

# The Python **socket** Module

A Complete Guide to Network Programming in Python

---

Covers TCP and UDP sockets, client/server patterns, blocking vs. non-blocking I/O, timeouts, common options, and troubleshooting — with runnable examples.

# Table of Contents

1. Introduction
2. Core Concepts
3. Creating a Socket
4. TCP: Client and Server
5. UDP: Client and Server
6. Sending and Receiving Data Reliably
7. Timeouts and Blocking Modes
8. Common Socket Options
9. Handling Multiple Clients
10. Closing Sockets Properly
11. Error Handling
12. Security Notes
13. Quick Reference Table

# 1. Introduction

The **socket** module gives low-level access to the BSD socket interface, letting Python programs communicate over networks using TCP, UDP, and other protocols. It underlies higher-level modules like **http.client** and **asyncio**.

This guide focuses on IPv4 TCP and UDP sockets, which cover the vast majority of real-world use cases.

```
import socket
```

## 2. Core Concepts

A socket is identified by an address family, a socket type, and is bound to a specific address (host and port).

- **Address family** — AF\_INET (IPv4) or AF\_INET6 (IPv6).
- **Socket type** — SOCK\_STREAM (TCP, connection-based, reliable) or SOCK\_DGRAM (UDP, connectionless, no delivery guarantee).
- **Address** — a (host, port) tuple, e.g. ('127.0.0.1', 8080).
- **Port** — an integer 0-65535; ports below 1024 usually require elevated privileges.

*TCP guarantees ordered, reliable delivery via a connection (think: phone call). UDP sends independent packets with no guarantee of delivery or order (think: postcards) — but with lower overhead.*

## 3. Creating a Socket

```
# TCP socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# UDP socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Recommended: use as a context manager to auto-close
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    ... # use s here
```

## 4. TCP: Client and Server

### TCP Server

```
import socket

HOST = "127.0.0.1"
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print(f"Listening on {HOST}:{PORT}")

    conn, addr = server.accept()
    with conn:
        print("Connected by", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            conn.sendall(data) # echo back
```

### TCP Client

```
import socket

HOST = "127.0.0.1"
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    client.sendall(b"Hello, server")
    data = client.recv(1024)

print("Received:", data.decode())
```

*bind() → listen() → accept() is the server-side sequence. connect() is all a client needs before sending or receiving data.*

## 5. UDP: Client and Server

### UDP Server

```
import socket

HOST = "127.0.0.1"
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
    server.bind((HOST, PORT))
    print(f"UDP server listening on {HOST}:{PORT}")

    while True:
        data, addr = server.recvfrom(1024)
        print(f"Received {data!r} from {addr}")
        server.sendto(b"ACK", addr)
```

### UDP Client

```
import socket

HOST = "127.0.0.1"
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as client:
    client.sendto(b"Hello, server", (HOST, PORT))
    data, server_addr = client.recvfrom(1024)

print("Received:", data.decode())
```

*UDP has no `bind()/listen()/accept()` sequence — there is no connection, so each datagram carries its own destination or sender address (`sendto` / `recvfrom`).*

## 6. Sending and Receiving Data Reliably

TCP is a byte stream, not a message stream — a single `send()` on one end may arrive as multiple `recv()` calls on the other, or vice versa. Never assume one `recv()` equals one `send()`.

```
# sendall() ensures all data is sent, unlike send()
sock.sendall(data)

# Reading until a fixed number of bytes is received
def recv_exact(sock, n):
    buf = b""
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("Socket closed early")
        buf += chunk
    return buf
```

For variable-length messages, define a simple framing scheme, such as sending a 4-byte length prefix before the payload.

```
import struct

# Sender
payload = b"some message"
sock.sendall(struct.pack("!I", len(payload)) + payload)

# Receiver
raw_len = recv_exact(sock, 4)
msg_len = struct.unpack("!I", raw_len)[0]
payload = recv_exact(sock, msg_len)
```

## 7. Timeouts and Blocking Modes

By default, sockets are blocking: calls like `recv()` or `accept()` wait indefinitely. This can be changed:

```
# Set a timeout (seconds) – raises socket.timeout if exceeded
sock.settimeout(5.0)

try:
    data = sock.recv(1024)
except socket.timeout:
    print("No data received in time")

# Fully non-blocking mode – calls raise BlockingIOError instead of waiting
sock.setblocking(False)

# Restore blocking mode
sock.setblocking(True)
```

*For handling many connections without threads, use the selectors module (built on `select()`/`poll()`/`epoll()`) together with non-blocking sockets, or use `asyncio` for a higher-level async API.*

## 8. Common Socket Options

```
# Allow immediate reuse of an address after the socket is closed
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# Disable Nagle's algorithm for lower latency (TCP only)
sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)

# Set send/receive buffer sizes
sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 65536)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 65536)
```

*SO\_REUSEADDR is almost always set on servers to avoid "Address already in use" errors when restarting quickly.*

## 9. Handling Multiple Clients

A basic server as shown in Section 4 handles only one client at a time. Three common approaches to serve multiple clients:

- **Threading** — spawn a new thread per connection. Simple, works well for moderate client counts.
- **selectors module** — a single thread multiplexes many non-blocking sockets. Scales well, more complex to write.
- **asyncio** — high-level coroutine-based networking, built on the same non-blocking model. Recommended for new async code.

### Threaded Server Example

```
import socket
import threading

def handle_client(conn, addr):
    with conn:
        print("Connected by", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            conn.sendall(data)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(("127.0.0.1", 65432))
    server.listen()
    while True:
        conn, addr = server.accept()
        thread = threading.Thread(
            target=handle_client, args=(conn, addr), daemon=True
        )
        thread.start()
```

*For production servers, prefer battle-tested frameworks (e.g. asyncio, Twisted) over hand-rolled threading loops.*



## 10. Closing Sockets Properly

```
# Always close sockets when done
sock.close()

# Preferred: context manager closes automatically
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
    ... # sock.close() called automatically on exit

# Gracefully signal no more data will be sent (half-close)
sock.shutdown(socket.SHUT_WR)
```

*shutdown() disables sending, receiving, or both, without releasing the socket's resources; close() releases the resources entirely. Using shutdown() before close() on TCP sockets helps ensure the peer sees a clean end-of-stream.*

## 11. Error Handling

```
import socket

try:
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(3.0)
    sock.connect(("example.com", 80))
except socket.gaierror:
    print("Hostname could not be resolved")
except socket.timeout:
    print("Connection attempt timed out")
except ConnectionRefusedError:
    print("Connection was refused")
except OSError as e:
    print("Socket error:", e)
finally:
    sock.close()
```

*socket.error is an alias for OSError in modern Python. Catch the most specific exception you can (gaierror, timeout, ConnectionRefusedError) before falling back to OSError.*

## 12. Security Notes

The socket module transmits data in plain text by default — anyone on the network path can read it.

- Use the **ssl** module to wrap sockets in TLS for encrypted, authenticated connections (`ssl.wrap_socket` / `SSLContext`).
- Never trust data received from a socket — validate lengths and content before parsing.
- Set sane timeouts on any socket accepting untrusted input, to avoid resource exhaustion from slow or stalled clients.
- Avoid binding servers to 0.0.0.0 unless the service is meant to be reachable from any network interface.

`socket.error` and unencrypted sockets are common sources of vulnerabilities — always prefer TLS for anything beyond local trusted testing.

## 13. Quick Reference Table

| Function / Method                                         | Purpose                                    |
|-----------------------------------------------------------|--------------------------------------------|
| <code>socket.socket(family, type)</code>                  | Create a new socket                        |
| <code>sock.bind((host, port))</code>                      | Bind socket to a local address             |
| <code>sock.listen()</code>                                | Mark a TCP socket as accepting connections |
| <code>sock.accept()</code>                                | Accept an incoming TCP connection          |
| <code>sock.connect((host, port))</code>                   | Connect to a remote TCP address            |
| <code>sock.send(data)</code> / <code>sendall(data)</code> | Send data over a TCP connection            |
| <code>sock.recv(bufsize)</code>                           | Receive data over a TCP connection         |
| <code>sock.sendto(data, addr)</code>                      | Send a UDP datagram to an address          |
| <code>sock.recvfrom(bufsize)</code>                       | Receive a UDP datagram and sender address  |
| <code>sock.settimeout(seconds)</code>                     | Set a timeout for blocking operations      |
| <code>sock.setblocking(bool)</code>                       | Toggle blocking / non-blocking mode        |
| <code>sock.setsockopt(...)</code>                         | Configure low-level socket options         |
| <code>sock.shutdown(how)</code>                           | Half-close sending, receiving, or both     |
| <code>sock.close()</code>                                 | Release the socket's resources             |

*End of guide.*